Introduction

In this lab, you will write and test code on the ESP32 that calibrates its analog-to-digital converter in order to read the voltage on the board's battery input. This will allow the board to track the state of charge of an attached battery.

WARNING!!!

This lab requires that you use the bench power supply. Make sure you connect the power supply to the Vbatt input on the board! Driving the "5V" input above 5 volts will damage the ESP32.

ESP32 Analog-to-Digital Converter and Dividing the Input Voltage

The ESP32's analog-to-digital converter (ADC) can nominally only convert voltages from 0 to 3.3 V. However, for this lab we assume that the battery voltage ranges from is 0 to 14.2 V, based on our use of 12 V motors. To reconcile this, we installed a voltage divider on the board that effectively applies a scaler of 10/(10+33) = 0.2272, as shown below.

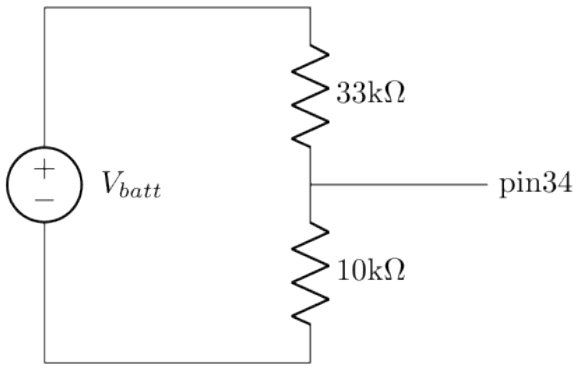

Figure 1: Voltage divider on class board

This effectively changes the range of input voltages from 0 to 14.2 V to 0 to 3.23 V. To convert a read voltage back to its original actual voltage before dividing, multiply it by 43/10. For example, if you read 2 V, the actual voltage value is 2 * 43/10 = 8.6 V.

Copyright 2025 Jason D. Bakos

Analog-to-Digital Conversion on ESP32

The ESP32 has a 12-bit ADC, so it returns a value from 0 to 4095 to represent input values of 0 V to 3.3 V. However, the ESP32's ADC is nonlinear, especially at the top and bottom ends of its range.

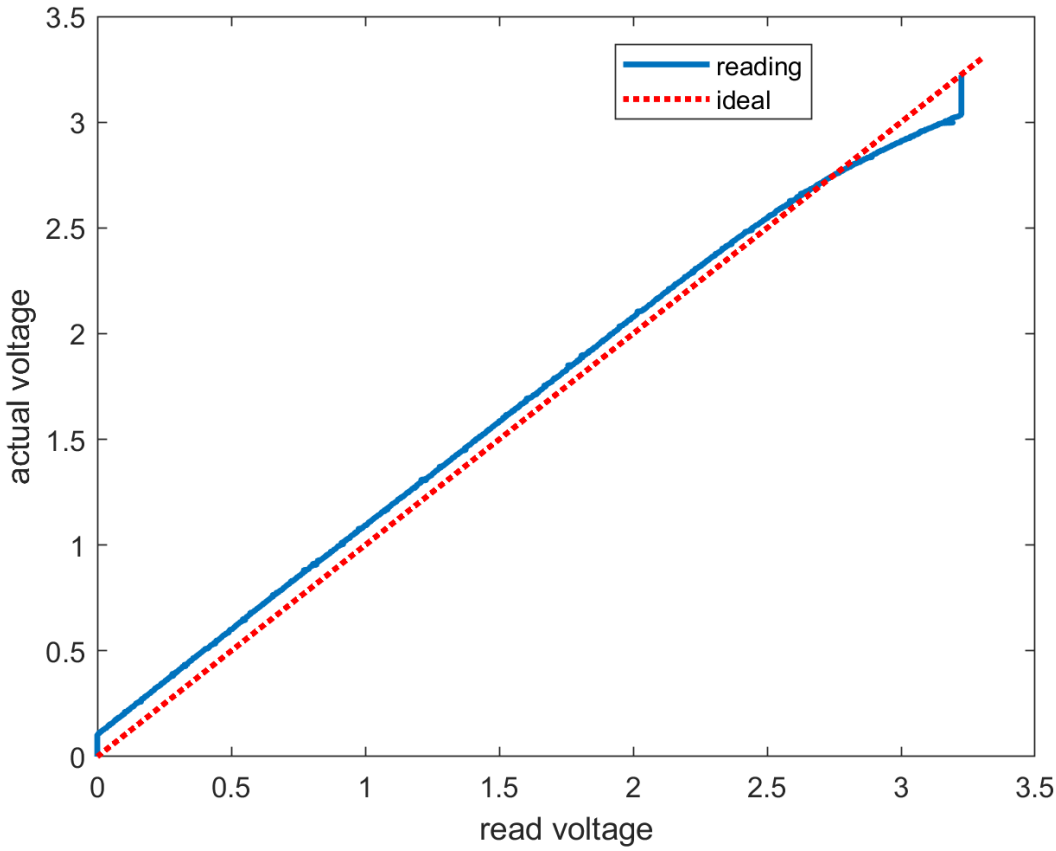

Figure 2: ADC Data

The goal of this lab is to autocalibrate the ADC, allowing the software to correct the ADC reading.

Use the following code to read the ADC value:

```
pinMode(15, INPUT);
int ADCValue = analogRead(34);
```

Calibration Procedure

To calibrate the ADC, you must find the coefficients for a function that converts the read ADC value into a estimation of the actual analog value, i.e `f(read_value)= real_value`. For this, we will use a piecewise linear approximation, in which the range of read values is divided into segments and each section will have its own linear approximation, i.e. a slope and intercept.

To build these functions, you will use a serial interface between the board and power supply, allowing the software on the ESP32 to force the power supply to sweep a range of output voltages and take an ADC measurement of each one.

UART on ESP32

The ESP32 has three hardware serial interfaces. Two are used by the USB interface and the SPI flash. The third one, U2UXD, is available for use. We can initialize this interface at 9600 baud (required by the power supply) with pin 12 as RX and pin 13 as TX using:

```
Serial2.begin(9600, SERIAL_8N1, 12, 13);
Serial2.print("<command>\r\n");
```

These pins are connected to the DB9 connector on the board. You can use a serial cable between this connector and the connector on the back of the lab power supply.

Power Supply Protocol

The Keithley 2231A-30-3 bench power supply supports remote control via a serial port, allowing us to automate the adjustment of the voltage being fed into the ESP32's ADC via the board's `Vbatt` input. The power supply communicates using a protocol called SCPI (Standard Commands for Programmable Instruments).

The first command that must be sent to the instrument is `*IDN?` which queries the power supply for identification. It will not respond to other commands until after this command is issued. Next, send the `SYST:REM` command to put the power supply in Remote mode. This locks out the front panel controls. After this, use the commands below to control the power supply. Send the `SYST:LOC` command to re-enable the front panel controls.

- `INST:NSEL X` : selects channel X [X=1 to 3]
- `VOLT X` : sets the voltage of the currently-selected channel to X volts
- `CURR X` : sets the current limit of the currently-selected channel to X amps

- `OUTP:ENAB X` : enables or disables the output of the current channel (X=1 to enable, X=0 to disable)
- `OUTP X` : enables or disables the output of all three channels (X=1 to enable, X=0 to disable)
- `SYST:BEEP` : beeps the PSU's speaker/buzzer
- `DISP:TEXT "Text String"` : Displays text on the power supply's display
- `DISP:TEXT:CLE` : clears text on the power supply's display and returns to showing the voltage/current readings

You can find the technical reference manual that contains the full command set at the following link: https://download.tek.com/manual/077100401_Reference%20manual.pdf

Calibration Model

For this lab we want to create a calibration model, which is a function `f(x)` in which `x` is the value read from the ADC and `f(x)` is the estimated actual analog value. When creating the model, you will control the power supply, so you will be able to create a set of `x` and `f(x)` pairs. In normal operation (e.g. measuring battery voltage), your code will only have access to the `x` value.

Piecewise Model

For this lab, create a piecewise simple linear model, in which there is a separate model created for different ranges of input values, as shown below, in which an exponential function is fitted to 5 linear pieces.

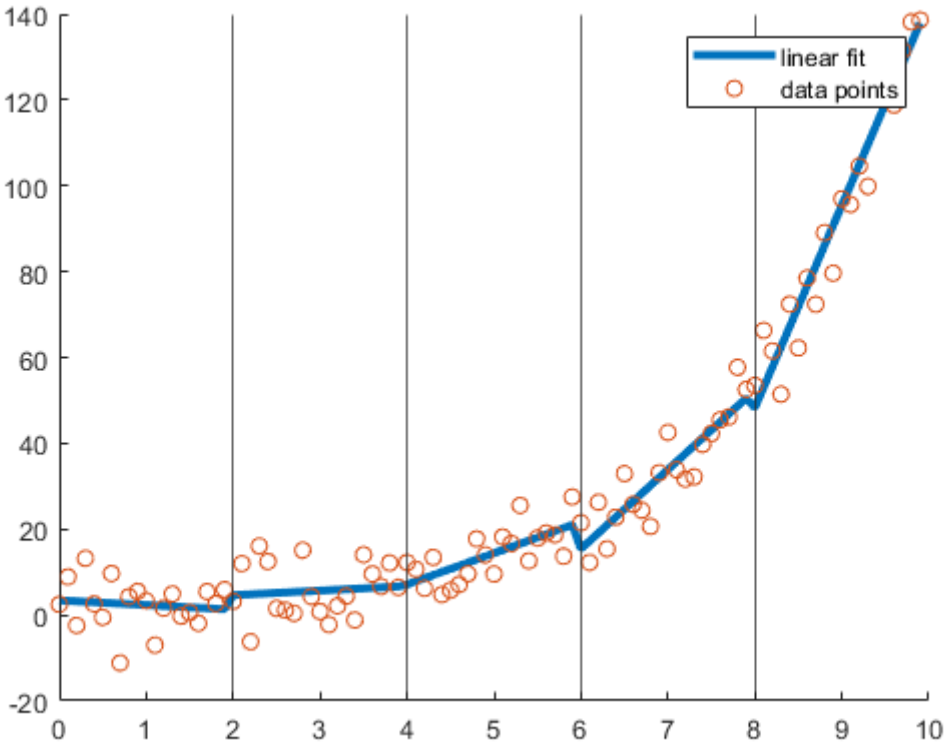

Figure 3: Piecewise model

In this lab, use 16 pieces, each covering 1/16 of the range of possible `x` values (from 0 to 4095).

Simple Linear Regression

Simple linear regression is a equation comprised of two parameters, a slope and an intercept, i.e. `f(x) = mx + b`. To find the values of `m` and `b` that minimizes the distance between each power supply value and the corresponding `f(x)`, you can use the following two equations:

$$m = \frac{n\sum_{i=1}^{n} x_i y_i - \sum_{i=1}^{n} x_i \sum_{i=1}^{n} y_i}{n\sum_{i=1}^{n} x_i^2 - \left(\sum_{i=1}^{n} x_i\right)^2}$$

$$b = \frac{\sum_{i=1}^{n} y_i \sum_{i=1}^{n} x_i^2 - \sum_{i=1}^{n} x_i \sum_{i=1}^{n} x_i y_i}{n\sum_{i=1}^{n} x_i^2 - \left(\sum_{i=1}^{n} x_i\right)^2}$$

Figure 4: Simple linear regression

Requirements

Your code should begin in “calibration mode”, in which it interfaces with the power supply and performs an auto-calibration by setting the power supply from 0 to 14.2 V in increments of 50 mV.

After auto-calibration is complete, your code should release control of the power supply and switch to “monitoring mode,” in which it continuously prints, every 1 second, its raw ADC value converted to a voltage, its calibrated ADC value, and its estimated Vbatt voltage including the voltage divider correction. This will allow a user to change the power supply voltage and observe the ESP32’s estimated value.

Example Output

```
Entering auto-calibration mode, please wait
Taking control of power supply...success
Autocalibrating
10% complete
20% complete
30% complete
40% complete
50% complete
60% complete
70% complete
```

```
80% complete
90% complete
100% complete
Entering monitoring mode
Raw voltage: 2.34 V [ADC read as 2904], corrected coltage: 2.54 V,
    actual voltage: 11.18 V
Raw voltage: 3.11 V [ADC read as 3859], corrected coltage: 3.21 V,
    actual voltage: 13.80 V
Raw voltage: 0.21 V [ADC read as 261], corrected coltage: 0.67 V,
    actual voltage: 2.88 V
```

Submitting Your Code

ZIP your code and submit it to Blackboard.

Rubric

(1) 10 points: demonstrate control of power supply
(2) 10 points: reading ADC value
(3) 10 points: perform calibration sweep
(4) 10 points: build piecewise model
(5) 10 points: demonstration of use of model

Copyright 2025 Jason D. Bakos